The Islamia University of Bahawalpur
Department of Information Technology

INFT-4041 · BSIT · 5th Semester · Fall 2026

E-Commerce Application Development

Chapter

8

Themes, Layout and the Customer's Experience

Part II — Standing the Shop Up

Taught in	Week 6
Laboratory	Page weight before and after, measured in bytes and requests
Assessment	**Assignment 1 set**

This is a class extract. It is *Chapter 8* of the unified handout, printed on its own so that a week's reading is a week's reading. Page numbers restart at 1 and are local to this extract; chapter, figure and table numbers are the ones the full volume gives them, so a reference you follow here lands in the same place there.

Cross-references to other chapters print as their numbers in the full volume. For the whole course in one file — front matter, the HEC tracing table, all twenty-three chapters and all seven appendices — see `ECAD_Fall2026_Handout.pdf`.

Dr. Mustafa Hameed · Department of Information Technology

Generated from `parts/` by `makechapters.py`; do not edit this PDF's source by hand

Chapter 8

Themes, Layout and the Customer's Experience

Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** how a theme is built from templates, and **override** one without editing the original.
- **Explain** what a layout is, and **place** a module on a chosen page without touching a template at all.
- **Measure** a page's weight and **say** which part of it is the weight.
- **Distinguish** the optimisations that are worth doing from the ones that are merely popular, using evidence.
- **Judge** a storefront change by its effect on the customer rather than by taste.

Before You Start

Chapter 6 and Chapter 7: a catalogue worth looking at. Appendix B's Twig section — this chapter edits templates. HTML and CSS from *Web Technologies*.

Key Terms in This Chapter

theme • template • Twig • template override • layout • module • position • page weight • request count • render-blocking • minification • responsive design • above the fold • conversion

8.1 What a Theme Actually Is

"Change the theme" sounds like changing a colour. It is not. A theme in OpenCart is a directory of Twig templates, and each one is responsible for one piece of the page.

```
docker compose exec shop \
  ls /var/www/html/catalog/view/template/product/
# category.twig  product.twig  search.twig  special.twig ...
```

The default theme lives in `catalog/view/template/`. The important rule about it is one you should adopt before you change a single line:

Never edit the default theme

Edit `catalog/view/template/product/category.twig` directly and three things happen. Your change is lost at the next upgrade. Nobody can tell what you changed, because there is no original to compare against. And a second developer, or you in March, will assume the file is stock and read it as documentation of how OpenCart works.

Make a **child theme** instead: a directory of your own that contains *only* the templates you changed. OpenCart looks there first and falls back to the default for everything else, so your directory *is* the list of your changes. A child theme with four files in it is a complete and honest statement that you changed four things.

8.2 Layouts, or How to Change a Page Without Editing It

A surprising amount of what people reach for a template to do can be done without touching one.

Layout, Module, Position

A **module** is a self-contained block — a category list, a banner, a carousel, "featured products".

A **layout** says which modules appear on which *kind* of page, and where. **Design › Layouts** lists them: Home, Category, Product, Checkout, and so on.

A **position** is a slot in the page: **Column Left**, **Column Right**, **Content Top**, **Content Bottom**. A module is placed in a position of a layout, with a sort order.

So "put a banner above the products on every category page" is three clicks in **Design › Layouts › Category** and no code at all.

Ask this before editing a template

"Is this a layout change or a template change?"

If you want something *moved, added or removed* and it already exists as a module, it is a layout change — do it in the admin panel, where it is visible to whoever comes next and survives upgrades.

Edit a template only when the *markup itself* must differ: a field shown that the theme does not show, a different arrangement within a block, a tag the theme gets wrong. Then override exactly that file in your child theme and nothing else.

The commonest waste of effort in this chapter's laboratory is a student editing `category.twig` to do something **Design › Layouts** would have done.

8.3 What a Page Weighs

Here the chapter stops being about arrangement. A customer's experience of your shop is mostly its speed, and speed is mostly *weight* — how many bytes and how many separate requests the browser must complete before the page is usable.

Note that this is a different question from Chapter 6's. That chapter measured how long the *server* took to produce the HTML. This one measures everything that happens afterwards, and it is usually larger.

Measured: What the Storefront Weighs

Produced by `code/m08_pageweight.py` on the machine in Appendix A. The harness fetches a page, finds every stylesheet, script and image the HTML names, requests each one once, and totals them.

Page	Requests	HTML	CSS	JS	Images	Total
Home	25	32 KB	348 KB	223 KB	293 KB	**895 KB**
Category	20	46 KB	348 KB	223 KB	122 KB	**738 KB**
Product	18	54 KB	354 KB	243 KB	61 KB	**713 KB**

The framework is the page. CSS and JavaScript come to about **570 KB on every page**, and they do not vary with the content — the same bytes on the home page, a category of 50,000 products and a single product. On the product page they are **84% of the weight**. The HTML that the whole of Chapter 6 was spent optimising is 6–7%.

Images are not the problem here, although everyone optimises them first. They are 33% of the home page, 16% of a category page and 9% of a product page. The largest single image in the demonstration catalogue is 13.9 KB. You could halve every image on the category page and save 61 KB — about 8%.

The single largest file is 265 KB of unminified CSS. `bootstrap.css` ships at 271,637 bytes, and **no minified version ships with it** — there is no `bootstrap.min.css` anywhere in the tree. A minified build of the same framework is around a tenth of that. That one file is 36% of the category page.

This measurement undercounts, and you should know how. It follows only assets the *HTML* names. Fonts are requested by the CSS, so they are invisible to it: the FontAwesome directory alone holds 248 KB of `.woff2` files, of which a browser fetches whichever subsets it needs. The totals above are therefore a floor, not a figure. A tool that drove a real browser would see more; this one is honest about seeing less.

The order to optimise in, which is not the popular order

From the numbers above, for this shop:

1. **Minify the CSS.** One file, 265 KB to roughly 27 KB. The largest single saving available, and it changes nothing a customer can see.
2. **Drop what you do not use.** Bootstrap and FontAwesome are general-purpose libraries; a shop uses a fraction of either. Removing the unused parts is the second-largest saving and the fiddliest.
3. **Cut the request count.** Twenty requests is twenty round trips. On a fast link this hardly matters; on a phone on a slow connection in Bahawalpur it is most of the wait.
4. **Then images** — resize to the size actually displayed, and serve modern formats.

Most advice reverses this list, because images are the thing people can see and a 2 MB photograph is a vivid story. On *this* shop images are 16% and the framework is 77%. On a shop with photographic product shots the balance moves. That is the point: the order depends on the measurement, and the measurement takes two minutes.

Render-blocking, and why position matters as much as size

A stylesheet in the `<head>` is **render-blocking**: the browser will not paint *anything* until it has fetched and parsed it. So 348 KB of CSS is not merely 348 KB of transfer — it is 348 KB standing between the customer and the first pixel.

Scripts block too, unless marked `defer` or `async`. A `<script src>` in the head stops the parser dead while it downloads.

This is why page weight and page time are not the same measurement, and why "it is only 200 KB" can still be the reason a shop feels slow.

8.4 Judging a Change by the Customer

The hardest discipline in this chapter is not technical. It is resisting the urge to change the storefront because you would prefer it differently.

</> Three changes, and how you would know if they helped

"Make the Add to Cart button green." You cannot know without an experiment: show half your visitors one colour and half the other, and compare the proportion who buy. With thirty visitors a day you will wait months for an answer, which is itself the answer — a shop this size should spend its effort elsewhere.

"Show the delivery charge on the product page." This you can reason about. Chapter 1's laboratory found that shops which hide the delivery charge until checkout lose customers there; it is the most-cited reason for abandonment. No experiment needed.

"Minify the CSS." Measurable directly, in bytes, with no customers involved at all. 265 KB to 27 KB is a fact, and it cannot make anything worse.

The pattern. Prefer changes you can verify without an experiment you cannot afford to run. Most small shops cannot generate enough traffic to measure a colour, and should spend their effort on the things that are measurable in bytes or known from everybody else's evidence.

Common Pitfalls

- **Editing the default theme.** Lost at upgrade, invisible to the next person. Override in a child theme.
- **Editing a template to do a layout's job.** If the block exists as a module, move it in **Design › Layouts**.
- **Optimising images first because they are visible.** On this shop that addresses 16% of the weight and ignores 77%.
- **Assuming a framework file is minified.** `bootstrap.css` here is 265 KB and there is no minified copy in the tree.
- **Measuring page weight by counting every `<img>` tag.** The same thumbnail on twenty cards is one request; counting it twenty times overstates the weight several-fold.
- **Forgetting fonts.** They are requested by CSS, so they are invisible to any harness that reads only the HTML — including this chapter's.
- **Redesigning on taste with no way to tell whether it helped.**

Try It Yourself: Weigh your shop, then make it lighter

```
# --- 1. Weigh three pages ------------------------------------------
cd code
python m08_pageweight.py --compare

# --- 2. Find the biggest single file -------------------------------
# Predict it before you look. Most people guess an image.
docker compose exec shop sh -c \
  'ls -laS /var/www/html/catalog/view/stylesheet/ | head -5'

# --- 3. Make a child theme -----------------------------------------
# Admin: Design > Theme Editor. Override ONE template -- the category
# product card -- and add the model number under each product name.
# Your override directory is now the list of your changes.

# --- 4. Do a layout change with no code ----------------------------
# Admin: Design > Layouts > Category. Add a module to Content Top.
# Reload a category page. No template was edited.
```

```
# --- 5. Weigh it again ------------------------------------------------
# Did your two changes move the number? By how much, and which one?
python m08_pageweight.py

# --- 6. Make a real saving --------------------------------------------
# Minify bootstrap.css and serve the minified copy instead. Any
# minifier will do. Record the before and after byte counts.

# --- 7. Weigh it a third time, and account for the difference -------
# The saving should be most of 240 KB. If it is not, find out what
# else changed -- do not just record the number.
python m08_pageweight.py

# --- 8. Say what you would do next, and why ---------------------------
# One paragraph, ordered by measured saving rather than by what is
# easiest to see. Chapter 20 returns to this with caching.
```

What to notice. Step 2 against step 1: almost everyone predicts an image and the answer is a stylesheet. That gap between where people look for weight and where weight is, is the whole chapter.

Checkpoint — Test Yourself

1. A product page is 713 KB, of which 597 KB is CSS and JavaScript. A colleague proposes compressing all the product photographs. What is the best case for that change, and what would you propose instead?
2. Distinguish a layout change from a template change, and give one example of each for the request "show a trust badge under the Add to Cart button on every product page".
3. This chapter's harness reports 738 KB for a category page, and a browser's developer tools report more. Give two reasons the harness is lower, and say which of them it could fix.

Chapter Summary

- A theme is a directory of Twig templates. Never edit the default: override only the files you change in a child theme, so that the directory is the list of your changes and survives upgrades.
- A layout places modules in positions on a kind of page. Moving, adding or removing an existing block is a layout change made in the admin panel, not a template edit.
- Measured: the storefront weighs 713–895 KB per page across 18–25 requests. CSS and JavaScript are about 570 KB of that *on every page* and do not vary with content — 84% of the product page.
- Images are 9–33% depending on the page, although they are what people optimise first.
- The largest single file is `bootstrap.css` at 265 KB, unminified, with no minified copy anywhere in the tree.
- The harness undercounts: it follows only assets named in the HTML, so fonts requested by CSS (248 KB in one directory) are invisible to it. Know what your instrument cannot see.
- Weight and time are not the same: a stylesheet in the head is render-blocking and stands between the customer and the first pixel.
- Prefer changes you can verify without an experiment you cannot afford to run. A small shop cannot measure a button colour and can measure 240 KB.

Review Questions

1. **(MCQ)** On the measured product page, CSS and JavaScript are what share of the weight? (a) 9%; (b) about a third; (c) 84%; (d) it depends on the catalogue size.

2. **(MCQ)** "Show the manufacturer's name under the product title" where the theme does not show it at all is: (a) a layout change; (b) a template override; (c) a module setting; (d) a database change.

3. **(Short)** Why is a child theme better than editing the default, giving two reasons that are not about upgrades.

4. **(Short)** Explain render-blocking, and why page weight and page time are different measurements.

5. **(Short)** This chapter's harness counts each unique asset once rather than once per mention. Explain what would go wrong otherwise, with a number.

6. **(Applied)** You are given a shop whose category page is 2.1 MB: HTML 40 KB, CSS 90 KB, JS 150 KB, images 1.82 MB across 40 requests. Give your optimisation order with a predicted saving for each step, and say how this differs from the order you would use on the shop in this chapter and why.

7. **(Applied)** Design a measurement that would tell you whether showing the delivery charge on the product page increases or decreases completed orders. State what you would record, how long you would need to run it, and — honestly — whether a shop taking thirty orders a day could reach a conclusion at all.